

L. D. SONAWANE DEGREE COLLEGE

Department of Computer Science

Bachelor of Science (Computer Science) — 2026

PROJECT REPORT

ON

“CAMPUS EXCHANGE”

Student Marketplace and Product Exchange Platform

Submitted By	Sujal Rai
Student ID	AF05215292
Batch Code	ANP-D4852
Course	B.Sc. Computer Science
Project Guide	Prof. Rajshri Thete
Development Stage	Sprint 1
Frontend	React + Vite
Backend	Python + FastAPI
ORM / Database	SQLAlchemy

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Science (Computer Science)

L. D. Sonawane Degree College

CERTIFICATE

This is to certify that the project report entitled “Campus Exchange — Student Marketplace and Product Exchange Platform” is a bonafide record of the project work carried out by Sujal Rai (Student ID: AF05215292, Batch Code: ANP-D4852), a student of B.Sc. Computer Science, L. D. Sonawane Degree College, during the academic year 2026, in partial fulfilment of the requirements for the award of the degree of Bachelor of Science in Computer Science.

The project report has been prepared under my guidance and supervision and, to the best of my knowledge, represents the independent work of the student for the Sprint 1 development stage of the project.

This project has not been submitted, in part or in full, for the award of any other degree or diploma of this or any other institution.

Rajshri Thete ma’am

Project Guide

DECLARATION

I, Sujal Rai (Student ID: AF05215292, Batch Code: ANP-D4852), hereby declare that the project report entitled “Campus Exchange — Student Marketplace and Product Exchange Platform” submitted for the Sprint 1 stage of my B.Sc. Computer Science programme at L. D. Sonawane Degree College is my own original work, carried out under the guidance of Prof. Rajshri Thete.

I further declare that this report, or any part of it, has not been copied from any other source without proper acknowledgement, and has not been submitted earlier for the award of any degree, diploma, or similar title of this or any other institution.

Sujal Rai

Student ID: AF05215292

Batch Code: ANP-D4852

INDEX

Sr. No. / Topic	Page No.
1. Certificate.....	2
2. Declaration.....	3
3. Acknowledgement.....	5
4. Abstract.....	6
5. 1. Introduction.....	7
6. 2. Objective.....	8
7. 3. System Analysis.....	9
8. 3.1 Problem Definition.....	9
9. 3.2 Preliminary Investigation.....	9
10. 3.3 Proposed System.....	9
11. 3.4 Feasibility Study.....	10
12. 3.5 Software Requirement Specification (SRS).....	11
13. 3.6 Functional Requirements.....	11
14. 3.7 Non-Functional Requirements.....	12
15. 3.8 Hardware and Software Requirements.....	12
16. 3.9 Software Engineering Paradigm.....	13
17. 4. System Design.....	14
18. 4.1 Data Flow Diagrams.....	14
19. 4.2 System Architecture.....	16
20. 4.3 Entity Relationship / Data Model.....	17
21. 4.4 Data Dictionary of Modules.....	18
22. 4.5 Module Design.....	21
23. 4.6 Procedural Design (Flowcharts).....	22
24. 5. Screenshots / User Interface.....	24
25. 6. Coding.....	28
26. 7. Testing.....	34
27. 8. Project Report / Results.....	36
28. 9. Future Scope.....	37
29. 10. Conclusion.....	38
30. 11. Bibliography & References.....	39

Acknowledgement

The successful completion of this project, “Campus Exchange — Student Marketplace and Product Exchange Platform,” would not have been possible without the support and guidance I received throughout its development.

I would like to express my sincere gratitude to my project guide, Prof. Rajshri Thete, for her constant encouragement, valuable suggestions, and technical guidance at every stage of this project. Her feedback on the application architecture, authentication flow, listing management, and transaction workflows was instrumental in shaping the final outcome of Sprint 1.

I am also thankful to the Department of Computer Science, L. D. Sonawane Degree College, for providing the resources, environment, and encouragement necessary to carry out this project as part of the B.Sc. Computer Science curriculum.

I extend my heartfelt thanks to my teachers, mentors, and classmates who provided their time for testing the application and offering constructive feedback that helped refine the user interface and overall user experience.

Finally, I am deeply grateful to my family and friends for their unwavering support, patience, and motivation throughout the duration of this project.

Sujal Rai
AF05215292 | ANP-D4852

Abstract

Campus Exchange is a student-focused digital marketplace designed to make it easier for students to buy, sell, exchange, and discover products within a campus community. The system replaces informal and fragmented methods of finding or transferring used products — such as scattered WhatsApp groups, notice-board flyers, and word-of-mouth — with a single, centralized web application built specifically around the needs of students.

The platform provides user registration and authentication, product listing creation with image support, category-based browsing, favorites, direct messaging between buyers and sellers, a structured buying workflow through Deals, a product-for-product Exchange workflow, nearby-seller discovery based on location, profile management, and a review system. Users can manage their own listings through the My Listings module and can remove a listing without physically deleting the underlying database record, preserving data integrity for future reference and analytics.

The system separates the frontend and backend into two independent applications. The frontend is developed using React and Vite, providing a fast, component-driven single-page application experience, while the backend uses Python, FastAPI, SQLAlchemy, and Pydantic to expose a REST-style API, validate incoming data, and persist information in a relational database. Authentication is handled using JSON Web Tokens (JWT); protected routes on both the frontend and backend require a valid, authenticated user before allowing actions such as posting a listing, sending a message, adding a favorite, buying, or requesting an exchange.

The buying workflow is modelled using a Deal entity that connects a buyer, a seller, and a listing, and progresses through Pending, Meetup, Completed, and Cancelled states with explicit confirmation flags for both parties. The exchange workflow is modelled using an Exchange entity that connects a requesting student, a receiving student, an offered listing, and a requested listing, and progresses through Pending, Accepted, Rejected, Completed, and Cancelled states. Together, these two entities give the marketplace a structured, auditable process for completing transactions instead of relying purely on informal chat-based agreements.

The project has been designed with modularity, maintainability, and future scalability in mind. The Sprint 1 repository is organized into two top-level directories — frontend and backend — each following a layered internal structure (models, schemas, services, routers on the backend; pages, layouts, and components on the frontend) that will allow the system to be extended with real-time chat, notifications, payments, and administrative tooling in later sprints.

1. Introduction

In the modern digital environment, web applications provide convenient, low-cost ways for people to discover information, communicate, and conduct transactions. This is especially true within a college campus, where hundreds of students go through the same academic cycle every year — buying textbooks in the first week of a semester, needing a scientific calculator for one course, borrowing lab equipment for a practical, or simply wanting to clear out stationery and gadgets they no longer use before graduating.

At present, most of this activity happens informally: through WhatsApp groups, printed notices pinned to a hostel board, or word-of-mouth between classmates. These channels work, but they are unstructured. There is no consistent way to search for a specific item, compare prices, verify a seller's identity, or keep track of whether a deal was actually completed. Items that could be reused within the same campus often go to waste simply because the right buyer and seller never discovered each other.

Campus Exchange addresses this gap by providing a dedicated, structured marketplace for a campus community. Instead of depending on informal groups or direct one-to-one messages with no context, students can publish structured listings that carry a title, description, price, condition, category, listing type, one or more images, and an availability status. Any other student on the platform can browse these listings, filter them by category or condition, and decide how they would like to proceed.

The application also recognizes that a student marketplace is not limited to direct cash purchases. Some students would rather trade one item for another — for example, exchanging a first-year physics textbook for a second-year mathematics textbook — instead of paying for or receiving cash. The system therefore includes two parallel transaction workflows: a purchase-oriented Deal workflow for conventional buying and selling, and an Exchange workflow for direct product-for-product trades.

Authentication is central to how the application protects its users. Actions such as posting an item, messaging a seller, adding a listing to favorites, buying a product, or requesting an exchange all require the user to be logged in. Any unauthenticated visitor who attempts one of these actions is redirected to the login page, ensuring that every transaction, message, and listing on the platform can always be traced back to a verified account.

This report documents the Sprint 1 implementation of Campus Exchange. It follows the conventional academic project-report structure — acknowledgement, abstract, introduction, objectives, system analysis, system design, screenshots, coding, testing, results, future scope, conclusion, and references — while adapting every section to describe the Campus Exchange application specifically, based on its actual React frontend and FastAPI backend implementation.

2. Objective

The overarching objective of Campus Exchange is to provide a simple, organized, and student-oriented platform for discovering, buying, selling, and exchanging products within a campus community. The specific objectives identified for the Sprint 1 stage of the project are as follows:

- Provide a centralized marketplace where students can post and browse products without depending on informal channels such as chat groups or notice boards.
- Allow students to create, update, and manage their own listings, including multiple product images per listing.
- Support both normal selling and product-for-product exchange workflows within the same application.
- Provide secure registration and login using token-based (JWT) authentication so that every protected action is tied to a verified user.
- Allow buyers and sellers to communicate directly through an in-app messaging system tied to a specific listing.
- Provide a favorites feature so that users can save products of interest and revisit them later.
- Support structured deals with explicit buyer and seller confirmation, rather than relying purely on informal chat-based agreement.
- Support exchange requests that can be accepted, rejected, completed, or cancelled by the participating students.
- Support optional location (latitude/longitude) information on listings so that a Nearby Sellers feature can surface geographically relevant products.
- Maintain a modular, layered frontend/backend architecture that can be extended in future sprints without large-scale rewrites.
- Lay the groundwork for a review system so that buyers and sellers can rate one another after a completed deal, supporting trust within the community.

3. System Analysis

3.1 Problem Definition

Students frequently need to buy, sell, or exchange used academic material — textbooks, handwritten notes, lab equipment, calculators, electronics, and general stationery — but currently rely on informal and fragmented communication channels to do so. These channels make it difficult to:

- Discover which items are currently available within the campus.
- Maintain consistent, structured information about a product (price, condition, category).
- Identify and contact a genuine seller without going through unrelated group chat noise.
- Track whether a transaction is still pending, in progress, or completed.
- Organize and respond to exchange requests separately from ordinary purchase requests.

Campus Exchange solves these problems by providing a structured web marketplace. Every product is stored as a Listing record with well-defined fields; every user is identified through an authenticated account; and every transaction — whether a purchase or an exchange — is represented through an explicit Deal or Exchange record with its own status, rather than being scattered across informal chat threads.

3.2 Preliminary Investigation

Before finalizing the scope of Sprint 1, a preliminary investigation was carried out to understand what a workable student marketplace would actually need. The investigation identified the following core requirements:

- Instant product discovery through a dedicated Browse Items interface with category filters.
- Structured product information, including price, condition, category, listing type (Sell/Exchange), and availability status.
- Secure, token-based user authentication for every protected action.
- Direct, in-app communication between buyers and sellers, tied to the specific listing being discussed.
- Two clearly separated transaction workflows — one for buying, one for exchanging.
- A user-controlled listing management area (My Listings) where a seller can review and remove their own posted items.
- A favorites mechanism so that interesting listings are not lost while a student decides whether to purchase.
- Optional location fields on each listing so that a future “nearby sellers” feature can be layered on top without changing the data model.

This investigation directly shaped the module boundaries used in the system design — authentication, listings, favorites, messaging, deals, exchanges, and profile — each of which maps to a corresponding set of backend routers and frontend pages in the actual implementation.

3.3 Proposed System

The proposed system is a full-stack web application with a clean separation between the presentation layer and the business/data layer. The React frontend is responsible for rendering pages, handling navigation

between them, collecting form input, and communicating with the backend over HTTP using Axios. The FastAPI backend exposes REST-style endpoints, validates every incoming request using Pydantic schemas, applies authentication and ownership checks, executes business logic inside dedicated service modules, and persists data through SQLAlchemy models.

Compared to the informal alternatives it replaces, the proposed system offers several concrete improvements:

- A single source of truth for every listing, instead of the same item being described differently across multiple chat messages.
- Explicit transaction states (Pending, Meetup, Completed, Cancelled for deals; Pending, Accepted, Rejected, Completed, Cancelled for exchanges) instead of ambiguous “is this still available?” chat threads.
- Ownership checks on every update or delete operation, so that only the student who created a listing can modify or remove it.
- A consistent authentication layer (JWT) shared by every protected feature, rather than each feature inventing its own notion of “logged in.”

3.4 Feasibility Study

A feasibility study was carried out across the five standard dimensions before development began, to confirm that the project was practical and achievable within the resources available for Sprint 1.

Feasibility Type	Assessment
Technical	React and Vite provide a modern, component-based frontend with fast development-time reloads. FastAPI provides a lightweight, high-performance Python API layer with automatic interactive (Swagger) documentation. SQLAlchemy provides ORM-based database access, and Pydantic schemas provide structured request/response validation. JWT authentication is a widely supported, stateless mechanism for protecting API routes.
Economic	All core technologies used (React, Vite, Python, FastAPI, SQLAlchemy) are open-source and free to use, which keeps the cost of development close to zero. A web-based architecture avoids the cost and complexity of a separate native mobile application during this initial development stage. Reusable service functions and React components reduce the amount of repeated development effort.
Operational	The interface is designed around marketplace actions that most students are already familiar with from consumer apps — browse, post, favorite, message, buy, and exchange — which keeps the learning curve low. Separating the frontend and backend into independent applications also makes day-to-day maintenance and debugging easier, since each layer can be tested in isolation.
Schedule	The Sprint 1 scope was deliberately divided into clearly bounded modules — authentication, listings, communication, deals, exchanges, favorites, and profile — so that each module could be implemented and tested independently within the sprint timeline.
Social	The system directly encourages reuse of products within the student community, which reduces unnecessary spending on items that a senior student may no longer need. Direct, listing-linked communication also helps students clarify product condition, price, and meetup details without exposing personal phone numbers to strangers.

3.5 Software Requirement Specification (SRS)

System Overview

Campus Exchange is organized into user-facing and service-oriented modules. The main functional areas identified for Sprint 1 are: authentication, listings, favorites, messaging, deals, exchanges, profile, nearby sellers, and reviews. Each of these areas corresponds to one FastAPI router, one SQLAlchemy model, and (where applicable) one or more React pages in the actual codebase.

System Modules

- User Module — registration, login, profile viewing and editing.
- Listing Module — creation, browsing, updating, and removal of marketplace products.
- Favorites Module — saving and removing listings of interest.
- Messaging Module — listing-linked conversations between two users.
- Deal Module — structured purchase transactions between a buyer and a seller.
- Exchange Module — structured product-for-product trade requests between two students.
- Review Module — post-deal ratings and comments between buyer and seller.

3.6 Functional Requirements

The following table summarizes the functional requirements identified for each module, based directly on the routers and services implemented in the backend.

Module	Functional Requirements
User	Register a new account; log in using email and password; view and update profile information (name, college, course, year, profile image); log out.
Listing	Create a listing with title, description, price, condition, category, listing type, and status; attach one or more images to a listing; browse all available listings; view a single listing's details; retrieve the current user's own listings; update a listing; remove (soft-delete) a listing owned by the current user.
Favorite	Add a listing to favorites; view the current user's list of favorited listings; remove a listing from favorites.
Message	Send a message to another user in the context of a specific listing; view the list of conversations; view and reply within an existing conversation; update or delete a previously sent message.
Deal	Create a deal for a seller's listing as the buyer; view deals the current user is involved in as buyer or seller; confirm participation in a deal (buyer_confirmed / seller_confirmed); update a deal's status as it moves toward completion; cancel a deal.
Exchange	Create an exchange request that connects an offered listing and a requested listing; view exchange requests sent or received; accept, reject, complete, or cancel an exchange.
Review	Create a review (rating and comment) tied to a completed deal; view reviews received by a user; update or delete a review.
Nearby Sellers	Retrieve listings near the current user's location using latitude/longitude and a configurable search radius.

3.7 Non-Functional Requirements

Requirement	Description
Security	Protected operations require a valid JWT access token, obtained after a successful login. Passwords are intended to be stored as hashes rather than plaintext. Ownership checks are performed before update or delete operations on listings, messages, and reviews.
Usability	The frontend provides a consistent left-hand navigation (Dashboard, Browse Items, My Listings, Nearby Sellers, Exchanges, Favorites, Messages, Deals, Profile) so that every major feature is reachable in a single click, with a clean, purple-accented visual theme.
Maintainability	Backend logic is separated into models, schemas, services, and routers, so that database structure, request validation, business logic, and HTTP endpoints can each be changed independently. The frontend similarly separates pages, layouts, and shared components.
Scalability	The frontend/backend separation, together with a stateless JWT authentication scheme, allows the two applications to be scaled or deployed independently as usage grows.
Reliability	Pydantic-based request validation and explicit, enumerated transaction states (for both Deal and Exchange) reduce the chance of the system entering an invalid or ambiguous state.
Performance	A REST-style API architecture with targeted SQLAlchemy queries (for example, filtering listings by the current user, or by proximity) keeps typical page-load operations fast even as the number of listings grows.
Portability	Because the frontend is a browser-based single-page application and the backend exposes a plain HTTP/JSON API, the system can run on any modern operating system with Python and Node.js installed, and can later be containerised for deployment.

3.8 Hardware and Software Requirements

Category	Requirement
Processor	Intel i5 or equivalent (development machine)
RAM	8 GB or more recommended
Storage	Sufficient free space for source code, dependencies, database files, and uploaded media
Operating System	Windows, Linux, or macOS
Browser	A modern browser such as Google Chrome, Microsoft Edge, or Mozilla Firefox
Backend Runtime	Python 3.x with FastAPI, SQLAlchemy, Pydantic, and python-jose (for JWT)
Frontend Runtime	Node.js and npm, with React, React Router, Vite, and Axios
Database	A relational database accessed through SQLAlchemy (SQLite for local development; extensible to PostgreSQL/MySQL for production)

3.9 Software Engineering Paradigm

The project follows a modular and iterative development approach, adapted from the traditional Waterfall model to allow feedback between phases. The Sprint 1 implementation can be viewed as one iteration of this adapted process: requirements are identified, the system is designed, individual modules are implemented, testing exposes issues, and the affected modules are refined before moving to the next sprint.

Development Phases

- Requirement Analysis — identify marketplace, authentication, listing, communication, deal, and exchange requirements based on real student needs.
- System Design — define frontend routes, backend modules, data models, entity relationships, and API contracts (request/response schemas).
- Implementation — build React pages and components on the frontend, and FastAPI routers, services, and SQLAlchemy models on the backend, in parallel.
- Testing & Debugging — validate individual workflows such as login, listing creation, listing removal, buying, and exchanging, and fix issues as they are discovered.
- Deployment & Maintenance — prepare the Sprint 1 repository (with separate frontend/ and backend/ directories) for continued development in future sprints and for eventual production deployment.

This iterative structure means that lessons learned while implementing later modules (for example, the Exchange workflow) were fed back into earlier modules (for example, tightening the ownership checks on Listing updates) rather than being treated as a strictly one-directional pipeline.

4. System Design

4.1 Data Flow Diagrams (DFD)

A Data Flow Diagram describes how information enters the system, is processed, and is stored. For Campus Exchange, the main external actor is the student/user, while the application contains authentication, listing management, messaging, favorites, deal, exchange, and profile processes, all of which ultimately read from and write to a shared database.

4.1.1 Zero-Level DFD

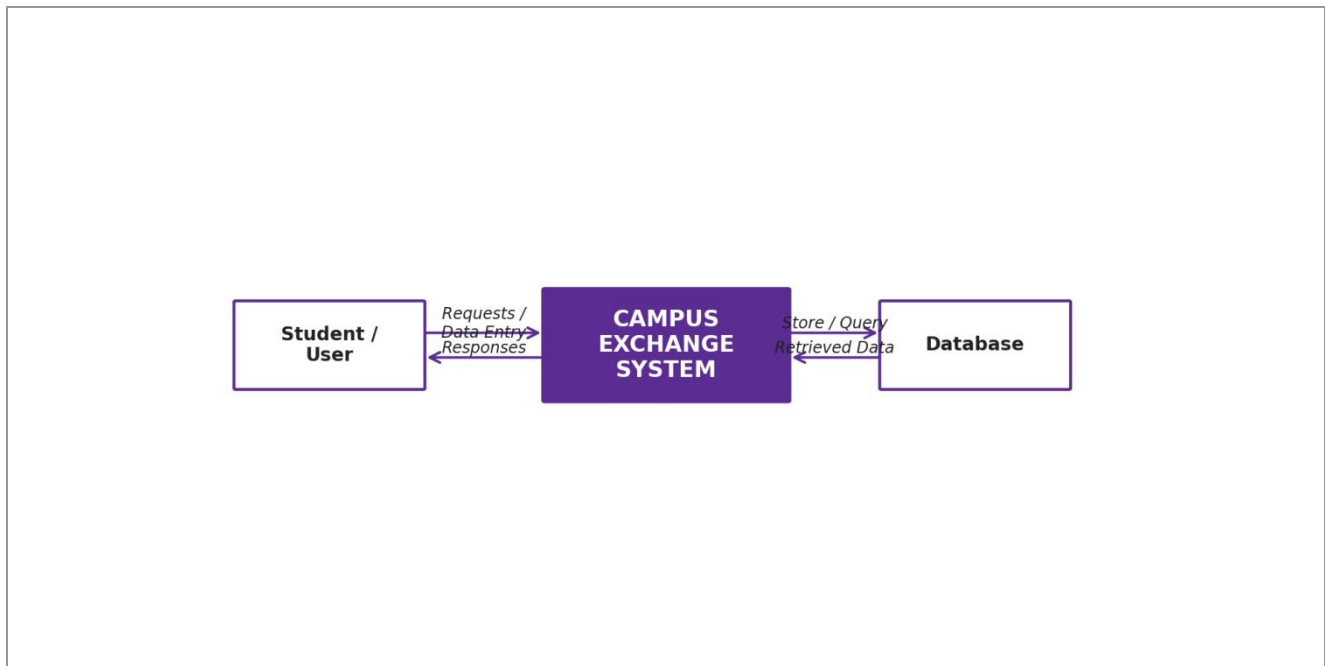


Figure 4.1: Zero-level data flow representation of Campus Exchange.

4.1.2 First-Level DFD

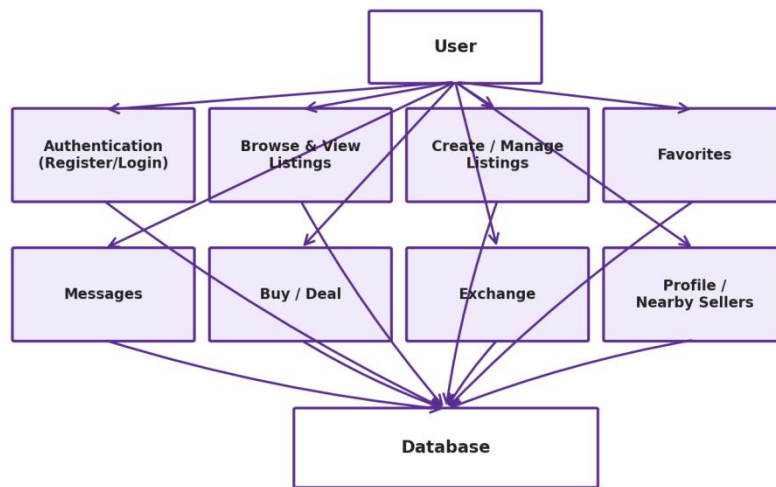


Figure 4.2: First-level flow of the major user-facing functions.

At the first level, the single “Campus Exchange System” process is expanded into its eight functional processes. Each of these processes corresponds directly to one FastAPI router and one React page group in the implementation: Authentication maps to /auth, Browse & View Listings and Create/Manage Listings both map to /listings, Favorites maps to /favorites, Messages maps to /messages, Buy/Deal maps to /deals, Exchange maps to /exchanges, and Profile/Nearby Sellers maps to /users and the location-aware listing query.

4.2 System Architecture

Campus Exchange follows a three-layer architecture that cleanly separates the user interface, the application/business logic, and the persisted data. This separation allows the frontend and backend to be developed, tested, and deployed independently.

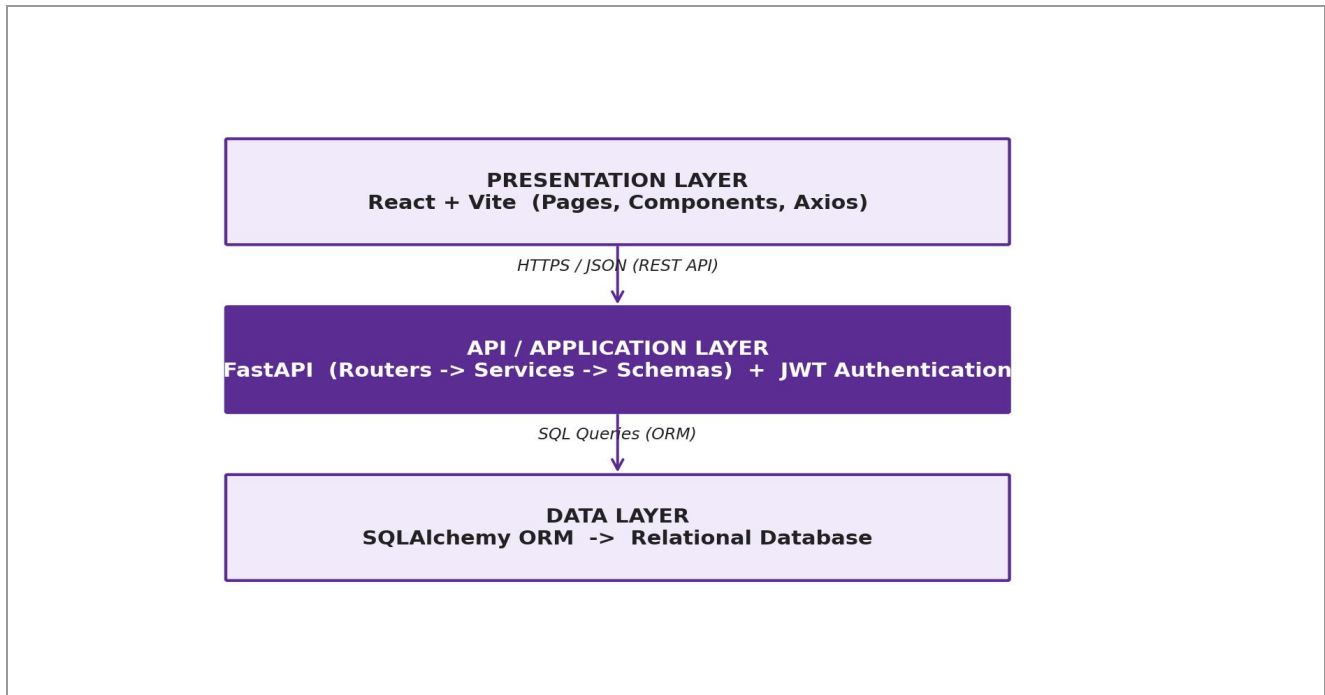


Figure 4.3: Three-layer application architecture of Campus Exchange.

The Presentation Layer is built with React and Vite and is responsible for rendering pages such as the Dashboard, Browse Items, My Listings, and Profile, and for sending HTTP requests to the backend using Axios. The API / Application Layer is built with FastAPI; every request first passes through a router, which delegates business logic to a service function, using Pydantic schemas to validate the shape of the incoming and outgoing data. JWT authentication sits at this layer as well — protected endpoints depend on a `get_current_user` function that decodes and verifies the access token before the request is allowed to proceed. The Data Layer uses SQLAlchemy models to represent each entity (User, Listing, Category, ListingImage, Favorite, Deal, Exchange, Message, Review) as a Python class mapped to a corresponding database table.

4.3 Entity Relationship / Data Model

The database is organized around users and the marketplace resources they interact with. A user can create multiple listings, save listings as favorites, send and receive messages, participate in deals as either a buyer or a seller, initiate or receive exchange requests, and write or receive reviews.

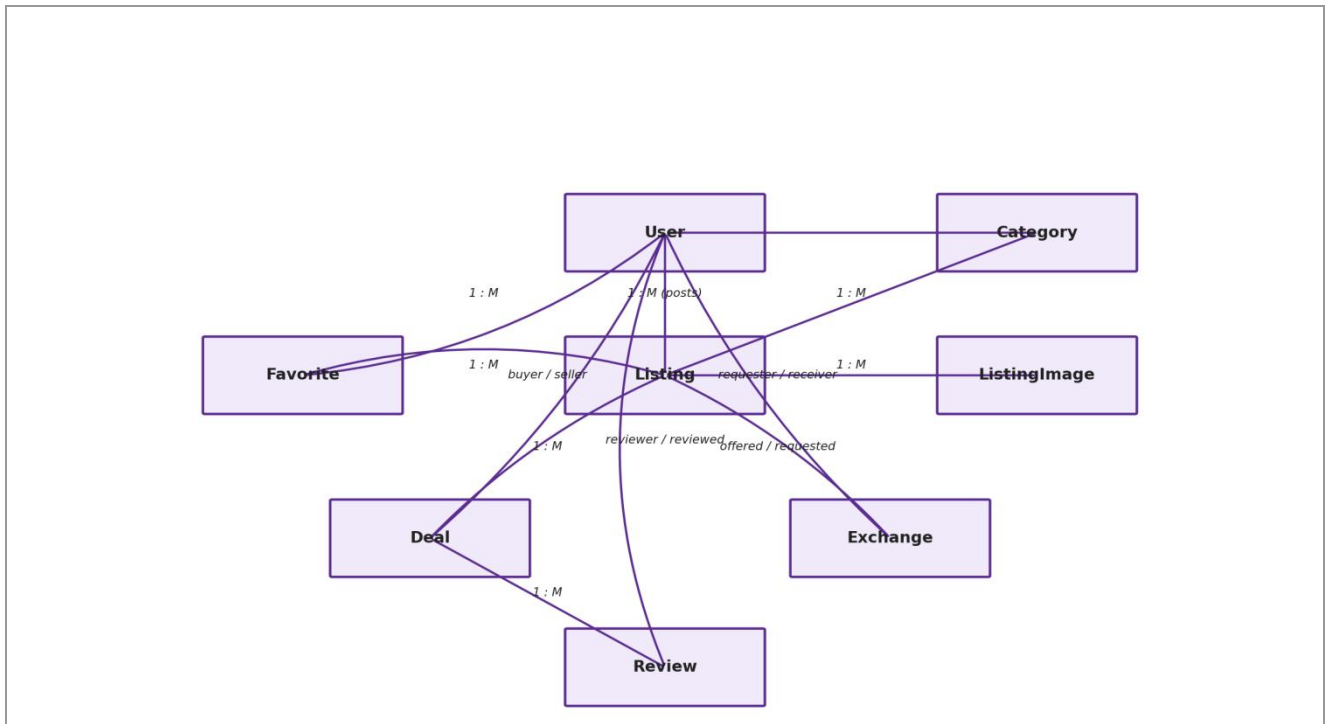


Figure 4.4: Simplified entity relationship model of Campus Exchange.

Each Listing belongs to exactly one Category and to the User who posted it, and can have any number of associated ListingImage records. A Favorite is a many-to-many join between User and Listing. A Deal references a single Listing and two distinct users (buyer and seller) via separate foreign keys. An Exchange references two distinct users (requester and receiver) and two distinct listings (offered and requested). A Review is tied to a completed Deal and records both the reviewer and the reviewed user.

4.4 Data Dictionary of Modules

The following tables describe the actual fields of each SQLAlchemy model implemented in backend/app/models, as used by Sprint 1.

4.4.1 User

Field	Type	Constraints / Notes
id	Integer	Primary key, autoincrement
name	String(100)	Not null
email	String(150)	Not null, unique, indexed
password	String(255)	Stored password hash, not null
college	String(150)	Not null
course	String(100)	Not null
year	Integer	Nullable — current year of study
profile_image	String(255)	Nullable — path/URL to profile picture
is_verified	Boolean	Default false
created_at	DateTime	Account creation timestamp

4.4.2 Category

Field	Type	Constraints / Notes
id	Integer	Primary key, autoincrement
name	String(100)	Not null, unique

4.4.3 Listing

Field	Type	Constraints / Notes
id	Integer	Primary key, autoincrement
user_id	Integer	Foreign key -> users.id (seller)
category_id	Integer	Foreign key -> categories.id
title	String(150)	Not null
description	Text	Nullable
price	Decimal(10,2)	Not null
condition	Enum	New / Like New / Good / Used
listing_type	Enum	Sell / Exchange, default Sell
status	Enum	Available / Sold / Exchanged / Removed, default Available
latitude	Decimal(10,8)	Nullable — for nearby-seller search
longitude	Decimal(11,8)	Nullable — for nearby-seller search
created_at	DateTime	Listing creation timestamp

4.4.4 ListingImage

Field	Type	Constraints / Notes
id	Integer	Primary key, autoincrement
listing_id	Integer	Foreign key -> listings.id
image_url	String(500)	Not null — path/URL of the image

4.4.5 Favorite

Field	Type	Constraints / Notes
id	Integer	Primary key, autoincrement
user_id	Integer	Foreign key -> users.id
listing_id	Integer	Foreign key -> listings.id
created_at	DateTime	Timestamp when favorited

4.4.6 Deal

Field	Type	Constraints / Notes
id	Integer	Primary key, autoincrement
listing_id	Integer	Foreign key -> listings.id
buyer_id	Integer	Foreign key -> users.id
seller_id	Integer	Foreign key -> users.id
meetup_location	String(300)	Nullable
status	Enum	Pending / Meetup / Completed / Cancelled, default Pending
buyer_confirmed	Boolean	Default false
seller_confirmed	Boolean	Default false
created_at	DateTime	Deal creation timestamp
completed_at	DateTime	Nullable — set on completion

4.4.7 Exchange

Field	Type	Constraints / Notes
id	Integer	Primary key, autoincrement
requester_id	Integer	Foreign key -> users.id
receiver_id	Integer	Foreign key -> users.id
offered_listing_id	Integer	Foreign key -> listings.id
requested_listing_id	Integer	Foreign key -> listings.id
status	Enum	Pending / Accepted / Rejected / Completed / Cancelled, default Pending
created_at	DateTime	Exchange request timestamp

4.4.8 Message

Field	Type	Constraints / Notes
id	Integer	Primary key, autoincrement
sender_id	Integer	Foreign key -> users.id
receiver_id	Integer	Foreign key -> users.id
listing_id	Integer	Foreign key -> listings.id
message	Text	Not null — message body (mapped from column 'message')
created_at	DateTime	Server-side default (current timestamp)

4.4.9 Review

Field	Type	Constraints / Notes
id	Integer	Primary key, autoincrement
deal_id	Integer	Foreign key -> deals.id
reviewer_id	Integer	Foreign key -> users.id
reviewed_user_id	Integer	Foreign key -> users.id
rating	Integer	Not null
comment	Text	Nullable
created_at	DateTime	Review creation timestamp

4.5 Module Design

Authentication Module

Registration creates a new User record with a hashed password. Login validates the submitted email and password against the stored hash and, on success, returns a signed JWT access token. The React frontend stores this token (in local storage) and attaches it as a Bearer token to every subsequent protected request.

Listing Module

The listing module manages creation, retrieval, update, and removal of products. Every listing belongs to exactly one seller (user) and one category, and can have any number of associated images. Listing removal is implemented as a status change to “Removed” rather than a hard delete, which keeps historical references (for example, from a completed Deal) intact.

Messaging Module

Messaging connects two users in the context of a specific listing. The frontend opens a conversation using the receiver's ID and the listing's ID, so that every message thread is naturally grouped by the product being discussed.

Deal Module

Deals represent conventional purchases. A deal record tracks the buyer, the seller, the listing, an optional meetup location, independent buyer_confirmed and seller_confirmed flags, and an overall status. A deal only becomes eligible for completion once both parties have confirmed.

Exchange Module

Exchanges represent product-for-product requests. An exchange record tracks the requester, the receiver, the offered listing, the requested listing, and a status that moves through Pending, Accepted, Rejected, Completed, or Cancelled as the two students negotiate the trade.

Favorites Module

Favorites allow a user to bookmark a listing for later review without committing to a purchase or exchange, implemented as a simple join table between users and listings.

Profile Module

The profile module allows a user to view and update their own account information — name, college, course, year, and profile picture — while keeping their email and username read-only to preserve account identity.

4.6 Procedural Design

4.6.1 User Panel Flow

A visitor can browse listings and view listing details without logging in. As soon as they attempt a protected action — favoriting a listing, messaging the seller, buying, or requesting an exchange — the application checks whether a valid token is present and redirects to the login/registration page if it is not.

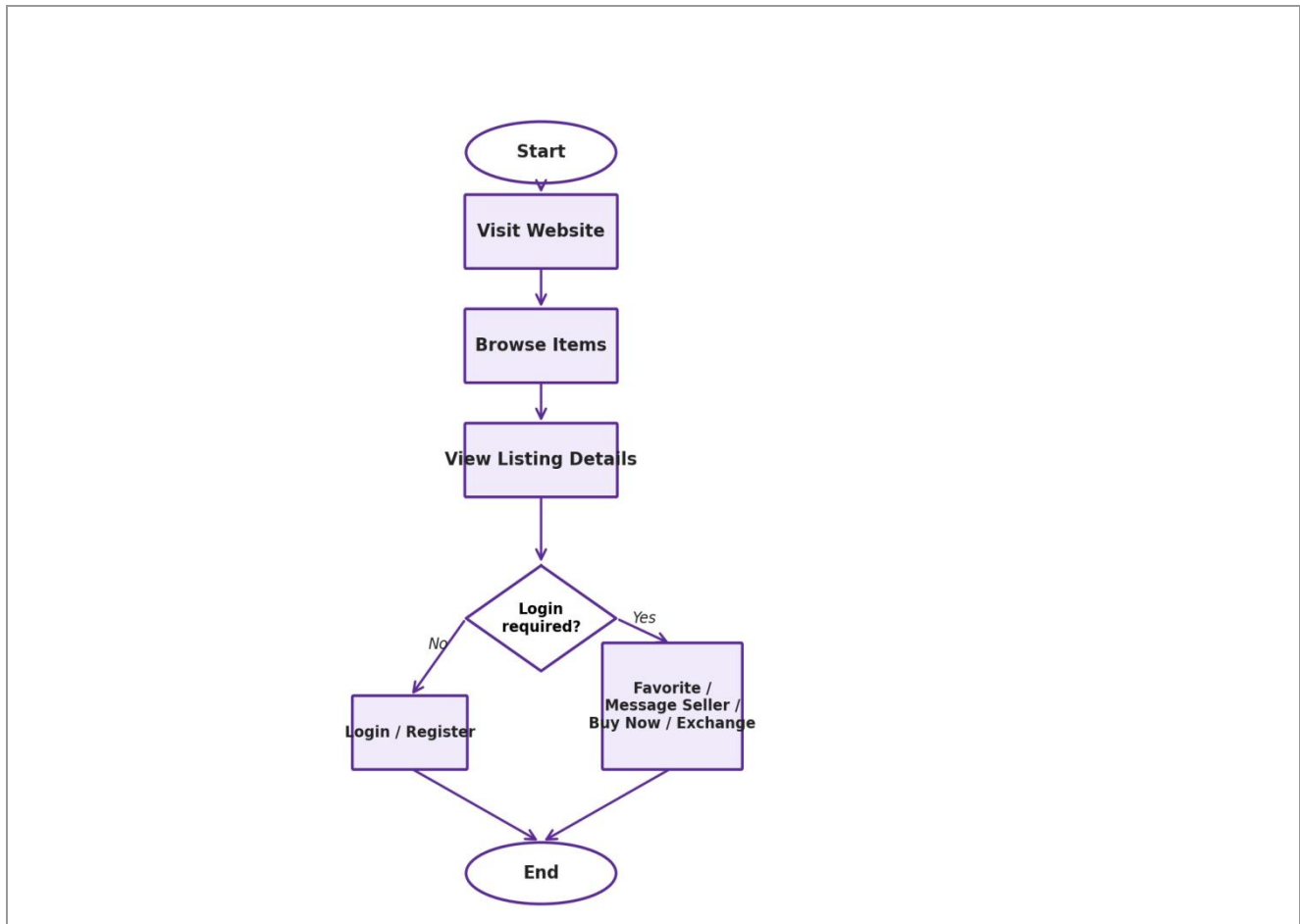


Figure 4.5: User interaction flow through Campus Exchange.

4.6.2 Authentication Flow

The authentication flow used by the login page submits the entered credentials to the `/auth/login` endpoint. If the credentials are valid, the backend returns a signed JWT access token, which the frontend stores and uses to redirect the user to the page they originally intended to reach (or the Dashboard by default). If the credentials are invalid, an inline error message is shown and the user can retry.

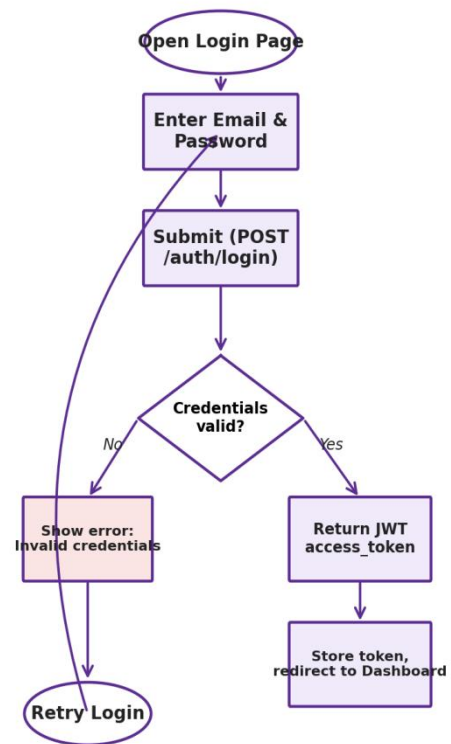


Figure 4.6: Authentication flow used by the application.

5. Screenshots / User Interface

The application uses a clean, light marketplace theme with purple accents. The main left-hand navigation is consistent across every page and contains Dashboard, Browse Items, My Listings, Nearby Sellers, Exchanges, Favorites, Messages, Deals, and Profile, with a persistent search bar, chat, notification, and account icon along the top bar. The following screenshots were captured directly from the running Sprint 1 build of Campus Exchange.

5.1 Dashboard

The Dashboard is the landing page after login. It welcomes the student, offers a prominent “+ Post Item” call-to-action, and surfaces the Popular Categories (Books, Notes, Lab Equipment, Electronics, Stationery, Others) as quick-access tiles, followed by a Recently Added section showing the latest listings posted by other students.

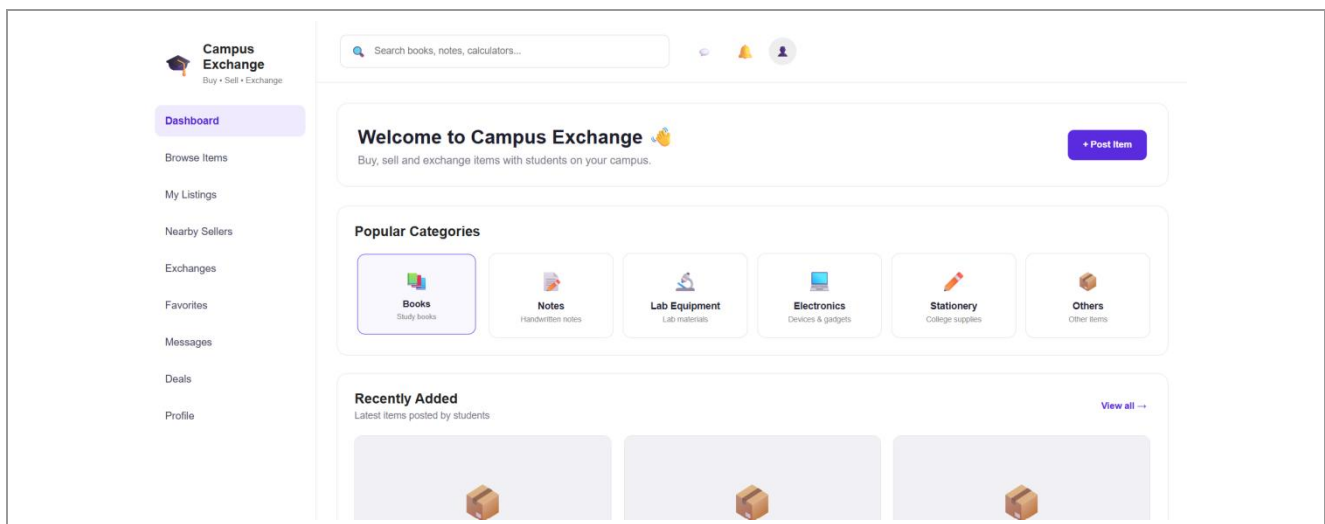


Figure 5.1: Dashboard page of Campus Exchange.

5.2 Browse Items

The Browse Items page is the main discovery surface of the application. Listings can be filtered by category (All, Books, Notes, Lab Equipment, Electronics, Stationery, Other) using the pill-style filter bar, and further refined using the Filters sidebar, which supports a text search within results, category radio filters, and a condition filter. Each result card shows the category badge, title, short description, condition, price, a favorite (heart) toggle, and a View button leading to the full listing details.

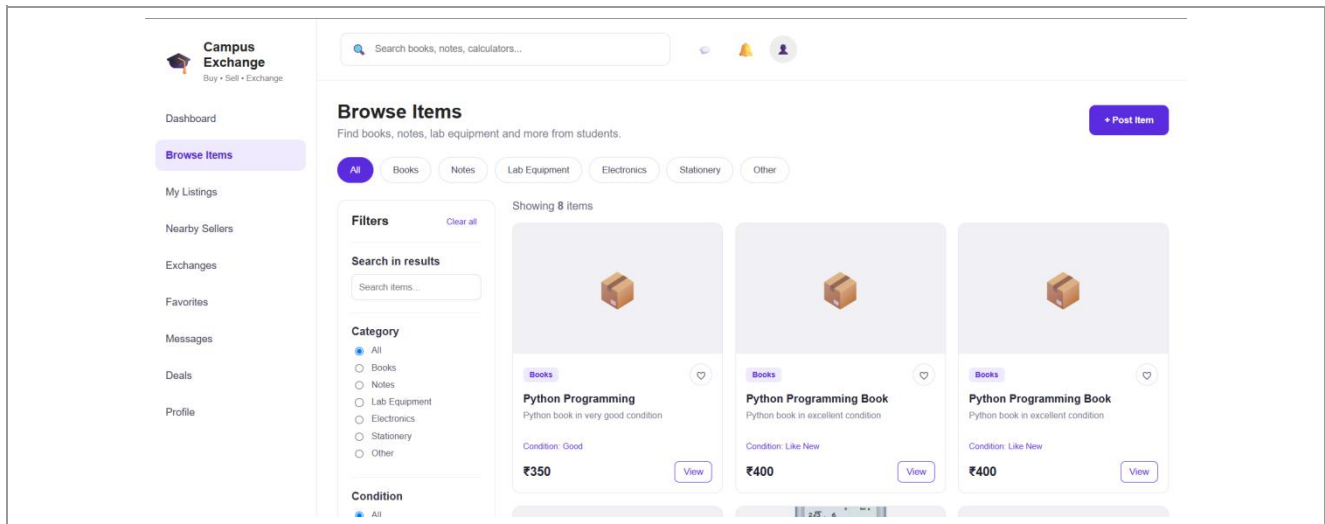


Figure 5.2: Browse Items page showing filterable listing results.

5.3 Nearby Sellers

The Nearby Sellers page uses the optional latitude/longitude fields stored on each listing to surface products from students located close to the current user. The page lets the student choose a search radius (for example, 10 km) and then requests location access before displaying nearby results on demand.

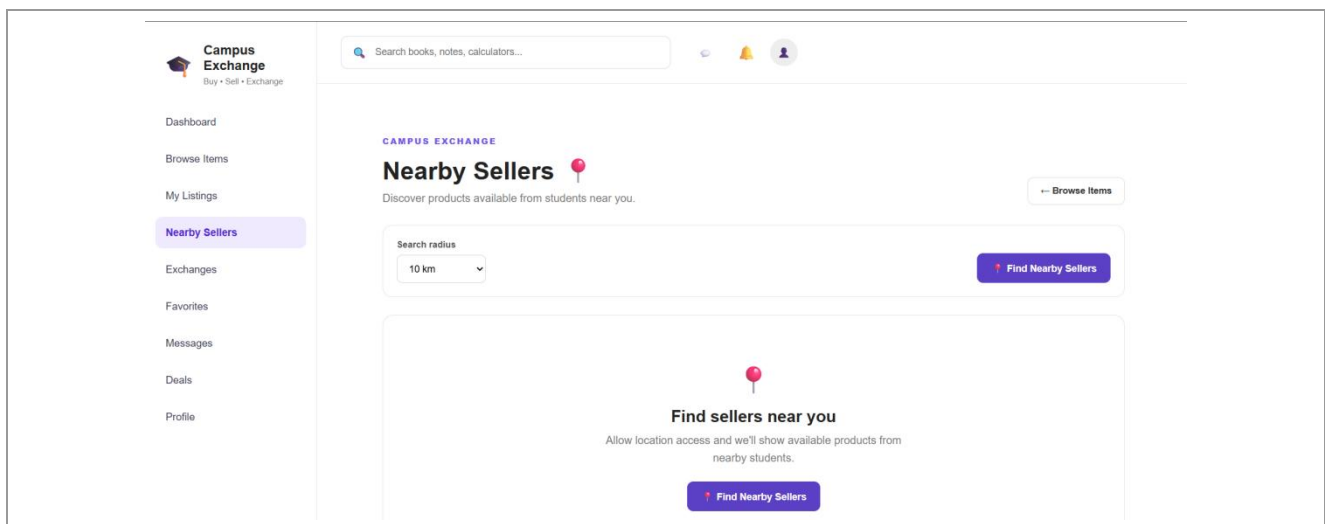


Figure 5.3: Nearby Sellers page with configurable search radius.

5.4 Favorites

The Favorites page lists every listing the current user has bookmarked, showing a saved-item card with a quick View Listing action and a Remove action to un-favorite the item. The page header also displays a running count of saved items.

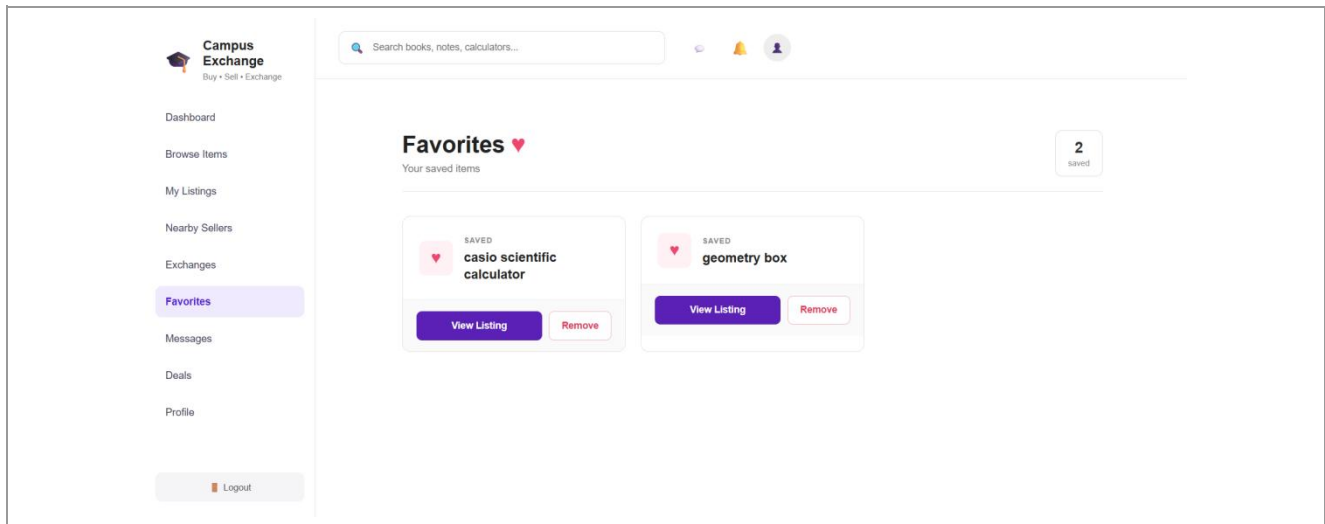


Figure 5.4: Favorites page listing the user's saved items.

5.5 Messages

The Messages page lists the student's active conversations, each grouped by the listing being discussed and showing the other participant's name and a preview of the latest message. A dedicated panel explains how to start a new conversation — by opening a product and choosing Message Seller — since conversations in Campus Exchange are always anchored to a specific listing rather than being free-standing chats.

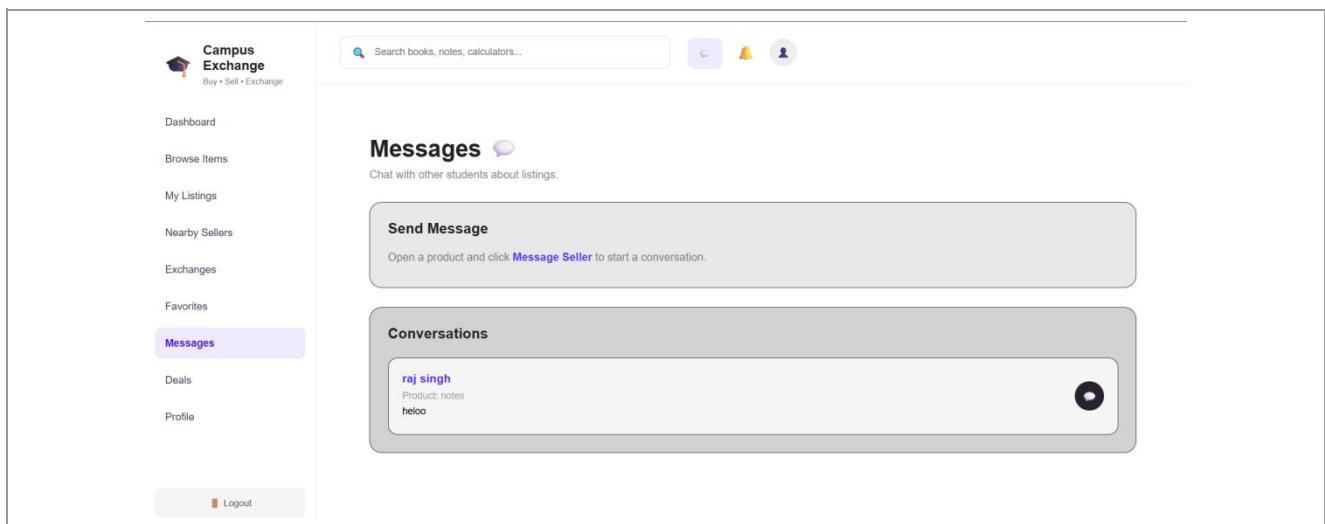


Figure 5.5: Messages page showing an active conversation.

5.6 Profile

The Profile page displays the logged-in student's account details — name, email, college, course, year, account ID, and verification status — along with an Edit Profile action for updating personal information and a profile picture.

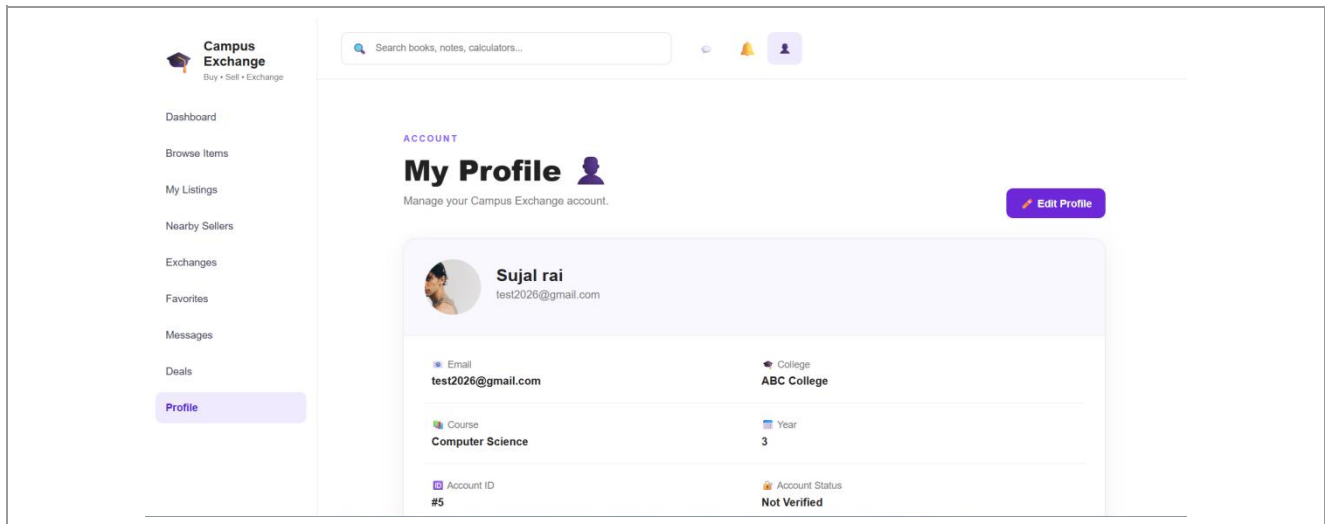


Figure 5.6: My Profile page showing account details.

5.7 Other Modules

The remaining modules identified in the system design — Login, Register, My Listings, Listing Details, Deals, and Exchanges — follow the same visual language shown above: a purple-accented action button, card-based content areas, and the same persistent left-hand navigation. My Listings mirrors the Browse Items layout but is filtered to the current user's own posted products and adds a Remove action on each card. Listing Details expands a single product into its full description, image gallery, seller information, and the Favorite / Message Seller / Buy Now / Propose Exchange actions described in Section 4.6.1. Deals and Exchanges each present the current user's transactions as a list of status-tagged cards (Pending, Meetup, Completed, Cancelled for Deals; Pending, Accepted, Rejected, Completed, Cancelled for Exchanges).

6. Coding

6.1 Backend Project Structure

The backend follows a layered design under `backend/app`. Models represent persistent SQLAlchemy entities, schemas validate incoming and outgoing API data using Pydantic, services contain the actual business/database logic, and routers expose that logic as HTTP endpoints.

```
backend/
|-- app/
|   |-- models/           (User, Listing, Category, ListingImage,
|   |                   Favorite, Deal, Exchange, Message, Review)
|   |-- schemas/         (Pydantic request/response models)
|   |-- services/         (business logic per module)
|   |-- routers/          (FastAPI endpoint definitions)
|   |-- database/         (SQLAlchemy engine + session)
|   |-- auth.py           (JWT creation & get_current_user)
|   |-- main.py           (FastAPI app, CORS, router registration)
|-- test_exchange.py
```

6.2 Frontend Project Structure

The frontend follows the standard Vite + React layout, with pages grouped by feature and a single `MainLayout` wrapping every authenticated route with the shared sidebar and top bar.

```
frontend/
|-- public/
|-- src/
|   |-- assets/
|   |-- layouts/          (MainLayout.jsx)
|   |-- pages/            (Dashboard, BrowseItems, Login, Register,
|   |                   ListingDetails, Messages, Deals, Favorites,
|   |                   Profile, CreateListing, Exchanges,
|   |                   NearbySellers, MyListings, Reviews)
|   |-- App.jsx           (route definitions)
|   |-- App.css / index.css
|   |-- main.jsx
|-- package.json
|-- vite.config.js
```

6.3 REST API Endpoint Summary

`main.py` registers ten routers on the FastAPI application, together exposing the following endpoints. This table was compiled directly from the router definitions in `backend/app/routers`.

Method	Endpoint	Purpose
POST	<code>/auth/login</code>	Authenticate a user with email + password and return a JWT access token.
GET	<code>/auth/me</code>	Return the currently authenticated user's profile.
GET	<code>/users/</code>	List all registered users.
GET	<code>/users/me</code>	Get the current user's own profile.
GET	<code>/users/{user_id}</code>	Get a specific user's public profile.
POST	<code>/users/</code>	Register a new user account.

Method	Endpoint	Purpose
PUT	/users/{user_id}	Update a user's profile information.
DELETE	/users/{user_id}	Delete a user account.
GET	/categories/	List all listing categories.
POST	/categories/	Create a new category.
GET	/listings/	Browse all available listings (with filters).
GET	/listings/my	Get the current user's own listings.
GET	/listings/nearby	Get listings near the current user's location.
GET	/listings/{id}	Get full details of a single listing.
POST	/listings/	Create a new listing.
PUT	/listings/{id}	Update a listing owned by the current user.
DELETE	/listings/{id}	Remove (soft-delete) a listing owned by the current user.
POST	/listing-images/	Attach an image to a listing.
GET	/listing-images/	List images for a listing.
DELETE	/listing-images/{id}	Remove a listing image.
POST	/favorites/	Add a listing to the current user's favorites.
GET	/favorites/	List the current user's favorited listings.
DELETE	/favorites/{listing_id}	Remove a listing from favorites.
GET	/messages/	List the current user's conversations/messages.
GET	/messages/{id}	Get a specific message.
POST	/messages/	Send a new message tied to a listing.
PUT	/messages/{id}	Edit a previously sent message.
DELETE	/messages/{id}	Delete a message.
GET	/deals/	List deals the current user is buyer or seller in.
POST	/deals/	Create a new deal for a listing.
PUT	/deals/{id}	Update / confirm a deal's status.
DELETE	/deals/{id}	Cancel a deal.
GET	/exchanges/	List exchange requests sent or received.
POST	/exchanges/	Create a new exchange request.
PUT	/exchanges/{id}	Accept, reject, complete, or cancel an exchange.
DELETE	/exchanges/{id}	Delete an exchange record.
GET	/reviews/	List reviews.
POST	/reviews/	Create a review for a completed deal.
PUT	/reviews/{id}	Update a review.
DELETE	/reviews/{id}	Delete a review.

6.4 Representative Code — JWT Authentication (backend/app/auth.py)

The auth module is responsible for creating signed access tokens at login time and for decoding/validating them on every subsequent protected request through the `get_current_user` dependency.

```
def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=ACCESS_TOKEN_EXPIRE_MINUTES
    )
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(security)
):
    token = credentials.credentials
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user_id = payload.get("sub")
        if user_id is None:
            raise credentials_exception
    except JWTError:
        raise credentials_exception
    ...
    return user
```

6.5 Representative Code — Listing Model (backend/app/models/listing.py)

```
class Listing(Base):
    __tablename__ = "listings"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    category_id = Column(Integer, ForeignKey("categories.id"))
    title = Column(String(150), nullable=False)
    description = Column(Text, nullable=True)
    price = Column(DECIMAL(10, 2), nullable=False)
    condition = Column(Enum("New", "Like New", "Good", "Used"))
    listing_type = Column(Enum("Sell", "Exchange"), default="Sell")
    status = Column(
        Enum("Available", "Sold", "Exchanged", "Removed"),
        default="Available"
    )
    latitude = Column(DECIMAL(10, 8), nullable=True)
    longitude = Column(DECIMAL(11, 8), nullable=True)
    created_at = Column(DateTime)

    user = relationship("User", back_populates="listings")
    category = relationship("Category", back_populates="listings")
    images = relationship("ListingImage", back_populates="listing")
```

6.6 Representative Code — Listing Removal (Ownership Check)

Listing removal follows the same ownership-then-soft-delete pattern used throughout the backend: the current authenticated user must own the resource before their request is allowed to change its status.

```
if listing.user_id != current_user.id:
    raise HTTPException(
        status_code=403,
        detail="Not authorized to remove this listing"
    )

listing.status = "Removed"
db.commit()
```

6.7 Representative Code — Frontend Login (frontend/src/pages/Login.jsx)

The Login page collects the student's email and password, posts them to the backend, and stores the returned JWT token in the browser so that subsequent requests can be authenticated.

```
const handleLogin = async (e) => {
  e.preventDefault();
  setError(""); setLoading(true);
  try {
    const response = await axios.post(
      "http://127.0.0.1:8000/auth/login",
      { email, password }
    );
    const token = response.data.access_token;
    localStorage.setItem("token", token);
    const redirectTo = location.state?.from || "/";
    navigate(redirectTo, { replace: true });
  } catch (error) {
    setError(error.response?.data?.detail || "Invalid email or password");
  } finally {
    setLoading(false);
  }
};
```

6.8 Representative Code — Application Routing (frontend/src/App.jsx)

App.jsx defines the client-side route table using React Router. Login and Register are rendered standalone, while every other page is nested inside MainLayout so it automatically receives the shared sidebar and top navigation bar.

```
<BrowserRouter>
  <Routes>
    <Route path="/login" element={<Login />} />
    <Route path="/register" element={<Register />} />

    <Route element={<MainLayout />>
      <Route path="/" element={<Dashboard />} />
      <Route path="/listings" element={<BrowseItems />} />
      <Route path="/nearby" element={<NearbySellers />} />
      <Route path="/my-listings" element={<MyListings />} />
      <Route path="/favorites" element={<Favorites />} />
      <Route path="/messages" element={<Messages />} />
      <Route path="/deals" element={<Deals />} />
      <Route path="/exchanges" element={<Exchanges />} />
      <Route path="/profile" element={<Profile />} />
    </>
  </Routes>
</BrowserRouter>
```

```
</Route>  
</Routes>  
</BrowserRouter>
```

7. Testing

Testing was carried out at multiple levels to verify individual modules, the interaction between modules, and complete end-to-end user workflows, following the standard progression from unit testing through to acceptance testing.

- Unit Testing — individual service functions (for example, a single listing-creation function or a single deal-status-update function) were tested in isolation.
- Module Testing — groups of related functions were tested together, such as the full set of listing management operations (create, browse, update, remove).
- Sub-system Testing — combined modules were tested together, such as authentication plus listings, to confirm that protected listing routes correctly rejected unauthenticated requests.
- System Testing — the complete application was exercised end-to-end, from a frontend button click through the API call to backend persistence and back to the UI.
- Acceptance Testing — the implemented Sprint 1 workflows were reviewed against the objectives defined in Section 2 to confirm they met the expected requirements.

7.1 Test Cases

ID	Test Case	Expected Result
TC01	Register with valid details	A new user account is created successfully and the user can subsequently log in.
TC02	Login with invalid credentials	Login remains unsuccessful and an “invalid email or password” error is shown.
TC03	Login with valid credentials	A JWT access token is returned and the user can access protected pages.
TC04	Create a listing with all required fields	The listing is stored and appears both in Browse Items and in My Listings.
TC05	Open My Listings	Only the current user's own listings are displayed, not other students' listings.
TC06	Remove own listing	The listing's status changes to “Removed” and it no longer appears in Browse Items.
TC07	Attempt to remove another user's listing	The operation is rejected with a 403 Forbidden response.
TC08	Add a listing to favorites	The listing appears in the Favorites page for that user.
TC09	Attempt to buy your own listing	The operation is rejected — a user cannot create a deal on their own listing.
TC10	Create a deal for another user's listing	A deal is created with the correct buyer, seller, and listing references, status Pending.
TC11	Confirm a deal as buyer or seller	Only the corresponding confirmation flag (buyer_confirmed / seller_confirmed) is updated.
TC12	Complete a deal after both confirmations	The deal status becomes Completed and the associated listing status becomes Sold.
TC13	Create an exchange request	An exchange request is created with the correct

ID	Test Case	Expected Result
		requester, receiver, offered, and requested listings.
TC14	Accept / reject / cancel an exchange	The exchange status updates accordingly and is reflected for both participants.
TC15	Send a message tied to a listing	The message appears in the Messages page for both sender and receiver, grouped under that listing.
TC16	Attempt any protected action while logged out	The user is redirected to the login page and the action is not performed.

8. Project Report / Results

The Sprint 1 implementation establishes the core marketplace workflow for Campus Exchange end-to-end, from user registration through to a completed purchase or exchange.

8.1 Implemented Areas

- Separate React (Vite) frontend and FastAPI backend applications, communicating over a REST/JSON API.
- User registration and login with JWT-based authentication.
- Protected API access — every sensitive route requires a valid access token.
- Product listing creation and management, including price, condition, category, and listing type.
- My Listings view with ownership-checked update and removal.
- Product image support via the ListingImage model and its dedicated router.
- Browse Items and Listing Details pages, including category and condition filters.
- Favorites for saving listings of interest.
- Listing-linked messaging between buyers and sellers.
- Deals for conventional purchases, with buyer/seller confirmation and status tracking.
- Exchange requests for product-for-product trades, with accept/reject/complete/cancel states.
- Nearby Sellers support using optional listing latitude/longitude.
- Profile management and a Review model to support post-deal ratings in a future sprint.

8.2 Transaction Result

The purchase workflow provides a clear, state-based process: a buyer creates a deal against an available listing, both buyer and seller confirm their participation, an optional meetup location can be arranged, and completion marks the deal Completed and the associated listing Sold. This removes the ambiguity of informal chat-based “is this still available?” exchanges.

8.3 Repository Result

The Sprint 1 repository is organized as two top-level applications — frontend and backend — each with its own dependency management (package.json for the frontend, a Python environment for the backend). This organization supports independent development of the two layers while keeping the entire application in a single repository for this stage of the project.

8.4 Security and Authorization

- Passwords are intended to be stored as hashes rather than plaintext.
- JWT tokens are required to authenticate every protected request.
- Listing ownership is checked before any update or removal is permitted.
- A user cannot create a purchase deal for their own listing.
- Exchange status updates are restricted to the two users directly involved in that exchange.

- Production secrets, such as the JWT signing key, should be moved out of source code and into environment variables before deployment.

9. Future Scope

The Sprint 1 build of Campus Exchange establishes the foundation of the marketplace. Several enhancements have been identified for future sprints:

- Real-time chat using WebSockets, so messages appear instantly instead of requiring a page refresh.
- Push/in-app notifications for new messages, deal confirmations, and exchange requests.
- Advanced search, filtering, sorting, and pagination on the Browse Items page.
- Cloud-based image storage and optimized image delivery for listing photos.
- Fully interactive, map-based nearby-seller search using geolocation.
- Optional payment integration for supported purchase workflows.
- Email verification at registration and a self-service password reset flow.
- An administrative dashboard for content moderation and marketplace oversight.
- Automated unit, integration, and end-to-end test suites, replacing the current manual test-case checklist.
- Production deployment with environment-specific configuration and a managed relational database.
- Analytics on listing views, completed deals, and exchange activity, to help the college understand marketplace usage.
- A fully surfaced Review and rating system, using the existing Review model, so buyers and sellers can build reputation over time.
- Improved accessibility and a responsive, mobile-first layout for use directly from a phone between classes.

10. Conclusion

Campus Exchange provides a structured digital marketplace that lets students buy, sell, exchange, and communicate about products without depending on informal, fragmented channels. The project turns ad-hoc product discovery and transaction activity into a single, centralized web application built specifically for a campus community.

The Sprint 1 implementation establishes the major foundations of the system: authentication, listing management, image support, browsing, favorites, messaging, deals, exchanges, profiles, and location-oriented functionality. The clean separation between the React frontend and the FastAPI backend provides a solid, maintainable base for continued development in future sprints.

The state-based Deal and Exchange workflows are particularly important, because they give the platform an explicit, auditable representation of transaction progress instead of relying solely on informal chat-based agreement. Ownership and authentication checks, applied consistently across listings, messages, and reviews, further support secure and trustworthy marketplace operations.

Overall, this project provides a practical and extensible base for a campus marketplace. It can be meaningfully enhanced in future sprints with real-time communication, notifications, payments, advanced search, an administrative dashboard, automated testing, and production deployment, while the core data model and API design established here remain stable enough to support all of these additions without a fundamental redesign.

11. Bibliography & References

11.1 Bibliography

- FastAPI official documentation and learning resources.
- React official documentation and learning resources.
- SQLAlchemy ORM documentation.
- Pydantic data-validation documentation.
- Python official documentation.
- Vite official build-tool documentation.
- PyJWT / python-jose documentation for JSON Web Token handling.

11.2 References

- FastAPI — <https://fastapi.tiangolo.com>
- React — <https://react.dev>
- Python — <https://www.python.org/doc>
- SQLAlchemy — <https://www.sqlalchemy.org>
- Pydantic — <https://docs.pydantic.dev>
- Vite — <https://vitejs.dev>
- React Router — <https://reactrouter.com>
- Axios — <https://axios-http.com>
- JSON Web Tokens (JWT) — <https://jwt.io>